

Search Typeahead System

HLD 101 — Project Report

Lakshay Jagga · Roll No. 24BCS10398

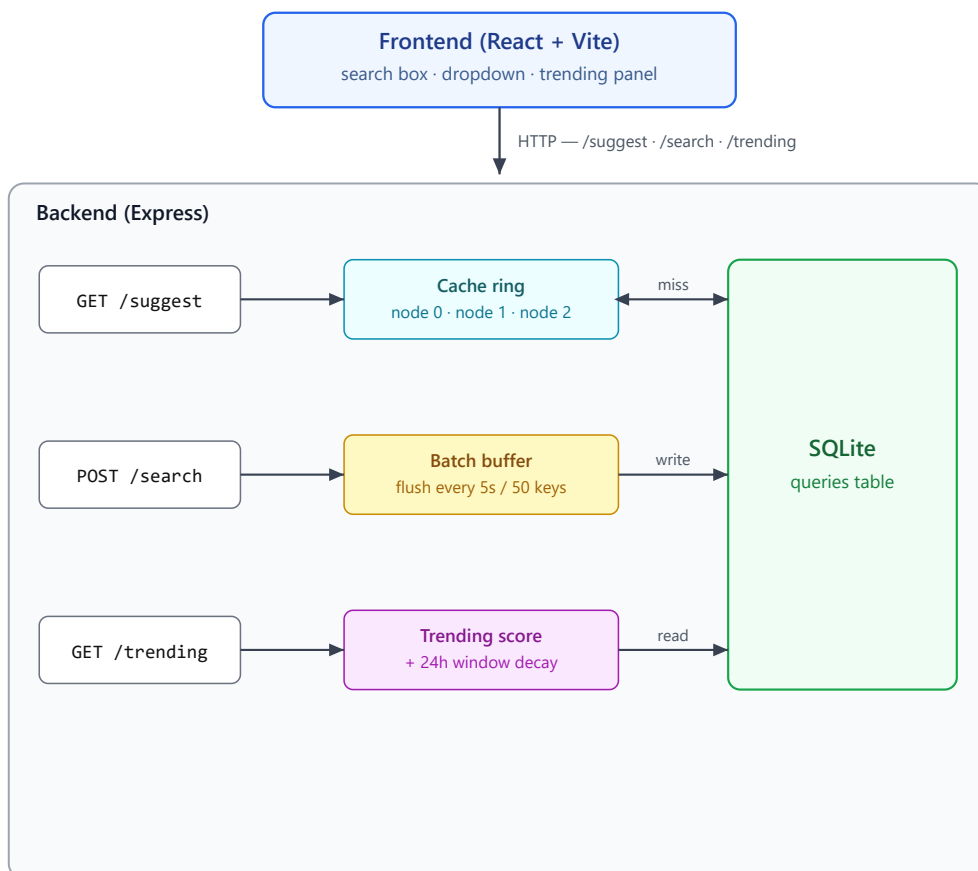
Overview

The goal of this project was to build a search suggestion system like the autocomplete you get on Google or an e-commerce site. As you type, it shows popular queries that start with what you've typed. When you actually run a search, that query's popularity goes up, and a trending list keeps track of what's been searched recently.

Most of the effort went into the backend, since that's where the interesting problems are: storing query counts, serving suggestions fast with a cache, splitting that cache across nodes with consistent hashing, and not writing to the database on every single search.

1. Architecture

There are two processes: a React frontend and a Node/Express backend. The frontend just talks to the backend over HTTP (Vite proxies the API calls during development). Everything interesting happens in the backend.



How a suggestion request flows

A `GET /suggest?q=goog` first hashes the prefix and checks the cache node it belongs to. If the prefix is cached, the result comes straight back from memory. If not, the backend runs a `LIKE 'goog%'` query against SQLite (sorted by count, limit 10), stores that result in the cache, and returns it. So the first request for a prefix is a database read, and repeats are served from memory until the entry expires (5 minutes).

How a search flows

A `POST /search` does **not** write to the database right away. It just adds the query to an in-memory buffer and returns immediately. A separate flush step writes the buffered counts to SQLite in one batch, either when 50 different queries have piled up or every 5 seconds. When a query is flushed, its cache entries are invalidated so the next suggestion read picks up the new count.

The cache "nodes"

The cache is three separate in-memory stores ("nodes"), and a consistent hash ring decides which node owns each prefix. In this project all three live inside one process, but the logic is the same as if they were three separate cache servers. This is the part that models how a real distributed cache would shard keys.

Main files

```
backend/
  server.js      Express app, mounts the routes, exposes /metrics
  db.js          SQLite schema + first-run seeding from the CSV
  cache.js       Consistent hash ring + the 3 node stores
  batchWriter.js In-memory buffer + flush logic
  trending.js    Trending score + 24h window decay
  routes/        suggest, search, trending, cacheDebug
  middleware/
    latency.js   Records p50/p95 latency per route
  scripts/
    ingestAol.js Loads the full AOL log
frontend/
  src/components/ SearchBox, SuggestionDropdown, SearchResult, TrendingPanel
  src/api.js      All the fetch calls
```

2. Dataset

Source

I used the **AOL search query log** (the `user-ct-test-collection-02.txt` file). It's a real log of searches from 2006. Each line is one search event with these columns:

```
AnonID    Query    QueryTime  ItemRank  ClickURL
```

There's no popularity count in the file. The popularity of a query is just how many times it appears in the log, so the loader counts occurrences.

After processing, file 02 has about **3.5 million search events** and **1.24 million unique queries**. The most-searched ones are `google` (32,396), `yahoo` (13,344) and `ebay` (12,949), which makes sense for 2006.

Because the data is from 2006, modern queries like "iphone" or "netflix" barely show up. To test the system, use queries that were popular then: `google`, `yahoo`, `ebay`, `mapquest`, `myspace`, `weather`, `lyrics`, and so on.

Loading

The raw log is about 217 MB, which is too big to put in a Git repo (GitHub rejects files over 100 MB). So I aggregated it once and committed the **top 100,000 queries** as a small CSV (`backend/data/seed.csv` , about 2 MB). The columns are simply `query,count` .

The backend seeds itself from that CSV the first time it starts, so there's no manual loading step:

```
cd backend
npm install
npm run dev          # first run seeds the DB from seed.csv, then reuses typeahead.db
```

If you want the full 1.24 million queries instead of the top 100k, put the AOL file in the project root and run:

```
npm run seed
```

That runs `scripts/ingestAol.js` , which streams the file line by line (so it never loads the whole 217 MB into memory at once), counts each query, and bulk-inserts the totals in a single transaction. It takes about 35 seconds.

3. API documentation

All endpoints are on `http://localhost:3001` .

GET /suggest?q= <prefix>

Returns up to 10 suggestions whose query starts with the prefix, sorted by count.

```
GET /suggest?q=goog

{
  "suggestions": [
    { "query": "google",      "count": 32396 },
    { "query": "google.com", "count": 8139 },
    { "query": "google earth", "count": 384 }
  ],
}
```

```
"source": "db",      // "db" on first lookup, "cache" on repeats
"node": 0           // which cache node served/stored it
}
```

Empty input returns `{ "uggestions": [], "source": "empty" }`. The prefix is lowercased, so `GOOGLE` and `google` give the same result.

POST /search

Body: `{ "query": "..."`. Records the search (into the batch buffer) and returns a confirmation. It does not write to the database synchronously.

```
POST /search { "query": "google" }

{ "message": "Searched" }
```

An empty query returns HTTP 400 with `{ "error": "query is required" }`.

GET /trending

Returns the top queries ranked by recency-weighted score. Optional `?limit=` (max 50).

```
GET /trending

{
  "trending": [
    { "query": "google", "count": 32396, "recent_count": 0, "score": 32396, ... },
    ...
  ]
}
```

GET /cache/debug?prefix=<prefix>

Shows which cache node owns a prefix and whether it's currently cached. Useful for showing the consistent hashing in action.

```
GET /cache/debug?prefix=google

{ "prefix": "google", "node": 0, "hit": false, "allMetrics": { ... } }
```

GET /metrics

One place to see latency, cache stats and batch-write stats. This is the endpoint I used for the performance numbers below.

```
GET /metrics
```

```
{
  "latency": { "/suggest": { "p50": "0.35ms", "p95": "0.78ms", "samples": 500 }, ... },
  "cache": { "hits": ..., "misses": ..., "hitRate": "96.9%", "nodeStoreSizes": [...] },
  "batchWrite": { "totalOpsReceived": 300, "totalDBWrites": 1, "writeReductionRatio": "99.7%" }
}
```

4. Design choices and trade-offs

SQLite with a prefix index

I used SQLite because the data fits easily and it needs no separate server. The query table has a unique `query` column with `COLLATE NOCASE` (so case doesn't matter) and an index on `query`. Suggestions use `LIKE 'prefix%'` rather than `LIKE '%term%'`. The trailing-wildcard form can use the index, while a leading wildcard can't, so this keeps prefix lookups fast even over 100k+ rows. The trade-off is that it only matches from the start of the query, so there's no fuzzy or typo matching.

Consistent hashing for the cache

The cache is split across 3 nodes. The naive way to pick a node would be `hash(prefix) % 3`, but that has a well-known problem: if you ever add or remove a node, the modulus changes for almost every key, so nearly the whole cache moves at once and you get a stampede of database reads.

Consistent hashing avoids this. Each node is placed at 150 points around a hash ring (the "virtual nodes" are just there to spread each node evenly so one node doesn't accidentally own a huge arc). To find a prefix's node, I hash the prefix and walk clockwise to the next node on the ring. Now if a node is added or removed, only the keys in that node's section move; the rest stay put. The cost is that it's more code than a modulus, and with only 3 nodes the split isn't perfectly even, which is exactly why the virtual nodes help.

Batch writes instead of writing on every search

Writing to the database on every search would mean one write per request, which is a lot of write pressure for what is really just a counter. Instead, each search adds to an in-memory buffer and the buffer is flushed in batches. I flush on two triggers: when 50 distinct queries have accumulated, or every 5 seconds, whichever comes first. The size trigger handles bursts and the time trigger makes sure nothing sits in the buffer forever during quiet periods.

The trade-off is durability. If the process crashes between flushes, the buffered counts are lost. For this project I decided that's acceptable, because these are popularity counters, not orders or payments, and losing a few seconds of counts doesn't really hurt. In a real system you'd protect the buffer with a write-ahead log or a durable queue (like Redis) before the database write. I also flush on Ctrl+C, so a normal shutdown doesn't drop anything.

Trending: blending all-time and recent

Trending uses $\text{score} = \text{count} + 10 * \text{recent_count}$, where `recent_count` only counts searches from the last 24 hours. Before ranking, any query that hasn't been searched in the last 24 hours has its `recent_count` reset to 0. That windowing is the important part: it means a query that went viral last week doesn't sit at the top forever. Once its recent activity ages out, it falls back to wherever its all-time count puts it.

One honest limitation: because the AOL all-time counts are huge (google is 32k), the top of trending is still mostly the big all-time queries unless something gets a lot of fresh searches. The recency weight nudges recent queries up, but it takes real volume to overtake the giants. I picked a weight of 10 as a reasonable middle ground; a higher weight would make trending react faster but also make it noisier.

A small optimization on trending

The straightforward way to rank trending is to compute the score for every row and sort, but that means sorting the whole table on every request. I avoided that. A query with no recent activity has a score equal to its plain count, so the top-by-count rows already cover all of those. The only other candidates are queries with `recent_count > 0`, which is a small set with its own (partial) index. So trending ranks just the top-by-count rows plus the recently-active ones, which is a few rows instead of all 100k. This dropped the trending p95 latency from about 33 ms to under 8 ms.

5. Performance report

These numbers come from the `/metrics` endpoint after warming up the cache and running a few hundred requests on a normal laptop (Windows, Node 24). The dataset for these runs was the committed 100,000-query seed.

Suggestion latency

	Latency
Cold read (cache miss, hits SQLite)	~1.5–6 ms
Warm read (cache hit)	~2 ms, and ~0.35 ms median under load
500 mixed requests	p50 0.35 ms, p95 0.78 ms, p99 7.5 ms

Under a tight loop the server handled about **1,475 suggest requests per second** on a single process. The cache hit clearly helps: a fresh prefix that has to hit the database is a few milliseconds, while a cached prefix is well under a millisecond.

Cache

After repeating a set of prefixes, the cache hit rate climbed to about **97%**, since the same prefixes keep getting reused. The keys were spread fairly evenly across the three nodes (for example 6 / 5 / 5), which is the consistent hashing plus virtual nodes doing their job.

Batch writes

In one run, **300 searches resulted in just 1 actual database write** (a 99.7% reduction), because those searches were spread over only a handful of distinct queries that got flushed together. In an earlier mixed run, 220 searches collapsed into about 18 writes (~92%). The exact ratio depends on how repetitive the traffic is and how it lines up with the flush timer, but in every case the number of database writes was far below the number of searches, which is the whole point.

Trending

After the optimization described above, `/trending` runs at about **p50 4.6 ms, p95 7.5 ms**, down from a p95 of roughly 33 ms when it was sorting the full table.

Summary

Component	Result
Suggest p95 (under load)	0.78 ms
Suggest throughput	~1,475 req/s
Cache hit rate (warm)	~97%
Write reduction	~92–99% fewer DB writes
Trending p95	~7.5 ms

The main takeaway is that the cache and the batch writer both do what they're meant to: reads are fast and mostly served from memory, and writes to the database are a small fraction of the number of searches.